# Yet Another Global Scheme (YAGS) implementation in gem5

EC513 - Spring 2026
Project Team: Brian Quijada, Fadi Kidess, Ozan Ekame Pekgoz, Will Borland

Abstract

In this work, we implement the YAGS branch predictor in Gem5. YAGS aims to minimize the destructive effects of branch address aliasing within the Pattern History Table (PHT) by adding caches that preserve prior executions of recent branches using 2-bit saturating counters and address tags. We evaluate our implementation over a set of benchmarks against 5 other branch predictors existing in Gem5. While our results are comparable to those of the paper (in terms of misprediction rates), we acknowledge several issues in our setup that warrant investigation for future work.

## Introduction

Branch prediction is crucial for modern pipelined processors, as stalling until a branch reaches execution would severely hurt performance. One main problem with global branch predictors is aliasing between two different indices inside the PHT. This aliasing can be both neutral, where these aliased entries contain the same information, or destructive, where the aliased entries contain different information. Destructive aliasing causes misprediction, which impairs processor performance.

In the paper we base our implementation on, the authors introduce five important prior designs they draw from and then describe the YAGS hierarchy and how it aims to reduce aliasing. The previous implementations are the following:

The Gshare scheme makes a simple change to the typical design of a global branch predictor by changing how the PHT is indexed. Rather than using only the concatenation of the branch address and branch history, Gshare uses the XOR of the branch address and branch history, which can lead to a more uniform use of entries in the PHT.

The Agree predictor uses the Branch Target Buffer (BTB) to store a bias bit. Since branches are heavily biased toward Not Taken (NT) or Taken (T), the bias bit is set at the beginning. Then, instead of using the usual NT/T, the PHT is selected using agree/disagree. This way, even if aliasing occurs, it is mostly neutral aliasing.

The Bi-mode has 3 PHT tables instead of 1 single PHT table. Two of the tables are direction tables, one labeled Direction NT and the other Direction T, and the third is a choice PHT table. As predictions proceed, we refresh only the NT or T Direction table corresponding to our guess. Eventually, this means that if aliasing does happen, we are indifferent to it, since it will be an overlap of NT with NT or T with T.

The skewed branch predictor is based on the principle that most of the aliasing that hurts global predictors stems from associativity conflicts rather than size conflicts. This design splits the PHT into three banks, increasing its associativity by using unique hash functions for each bank, without requiring expensive tags. These three banks aim to reduce aliasing in general and therefore destructive aliasing. The main tradeoff is that there is significant redundancy among the banks, increasing size conflict while decreasing aliasing.

The filter mechanism aims to reduce the amount of redundant information stored in the PHT. This is done by adding a saturating counter for each BTB entry, with the most significant bit (MSB) set to whatever direction the branch first takes. In the event a future branch takes the opposite direction to its bias, the counter is set to zero, and the bias bit is flipped. Otherwise, the counter increments, showing that the branch has a strong bias. If the branch has a strong bias, the BTB entry is directly used to predict the branch; otherwise falling through to the PHT. This significantly reduces information stored in the PHT, and, therefore, significantly reduces all aliasing.

## Branch Predictor Description

YAGS builds on the observation that for any given branch, most of its occurrences follow a dominant bias; either mostly taken or mostly not taken. Rather than storing all branch outcomes in one large PHT like Gshare, YAGS splits prediction into two components.

The first component is a choice PHT, a standard bimodal predictor indexed by the branch address XORed with the global history. This captures each branch's dominant bias and serves as the default prediction.

The second component is two small direction caches: a taken cache and a not-taken cache. These store only the exceptions; the rare cases where a branch does not follow its dominant bias. Each entry in these caches contains a tag composed of the least significant bits of the branch address, which virtually eliminates aliasing between branches sharing the same cache slot.

When a branch occurs, the choice PHT is consulted first. If it predicts taken, the not-taken cache is checked for a tag match, because if this branch is in the not-taken cache, it means this is one of the rare times it defies its taken bias. If there is a cache hit, the cache overrides the choice PHT prediction. If there is a miss, the choice PHT prediction stands.

This design reduces aliasing in two ways: the choice PHT handles biased branches cleanly, and the direction caches use tags to prevent different branches from interfering with each other's exceptions."

## Implementation and Testing

Our gem5 YAGS implementation includes the PHT and both T and NT caches implemented as vectors of saturating counters. The PHT and both caches have a parameterizable depth, which was set to 8,192 for our experimentation. Each cache is associated with a tag, which is the lowermost bits of the PC address. The tag is implemented as an unsigned vector. The tag's number of bits is a configurable parameter, which we set to 13 bits (the base-2 logarithm of 8,192).

The comparison logic and branch prediction are implemented in the yags.cc file, with the class and variable declarations being set in the yags.hh file. To add the predictor to the build process, we added the source files to the SConscript file and created a new ConditionalPredictor object for our YAGS implementation.

For reference, in our implementation, we shift the branch PC address by instShiftAmt, a gem5 built-in parameter. Since the PC might be 4-byte aligned, the 2 LSBs of the PC address will always be zero. Therefore, 75% of the cache may not be used at all. As a result, shifting by instShiftAmt yields the PC address so that the next address is incremented by 1 rather than 4.

We initially encountered segmentation faults when running our YAGS implementation due to incorrect input masking for the caches. To counter this, we implemented modulo indexing, in which only the 13 LSBs are used to index into the caches.

Evaluation Results

To evaluate our implementation against the branch predictors we had previously examined in our homework assignments, we needed to run simulations. Our original plan was to run simulations with 5 billion warmup instructions and 1 billion measurement instructions. This is important because it gives the branch predictor time for its structures, such as history tables or directional caches, to fully saturate and reach a steady state where measurements best reflect their operational capabilities. Unfortunately, there were a few flaws in the setup of our configuration and SCC submission scripts, which resulted in stats.txt files from these initial simulations being overwritten and not properly recorded. These flaws are further discussed in the conclusion. Because these simulations failed to produce results, we did not have enough time to rerun them with an order of 5 billion warmup instructions. The consequences of that, as well as flaws we recognized in our final simulations, will be discussed again in the conclusion.

Shown in Figure 1 are the misprediction rates for a set of benchmarks we evaluated with YAGS and other branch predictors. The misprediction rates for YAGS ranged from 5.3% to 6.5%. Figure 2 shows the IPCs for the same benchmarks and branch predictor set. The IPC ranges from 0.361 to 0.363, which raised to us a point of concern for us that we discuss in the conclusion below.

Compared with the paper's results, we achieve similar misprediction rates. Since we implemented 6 KB caches, we compare with the accuracy (accuracy rate = 100% - misprediction rate) of the paper's implementation at roughly the same cache size, and both works achieve roughly 6% misprediction rate (94% accuracy rate).

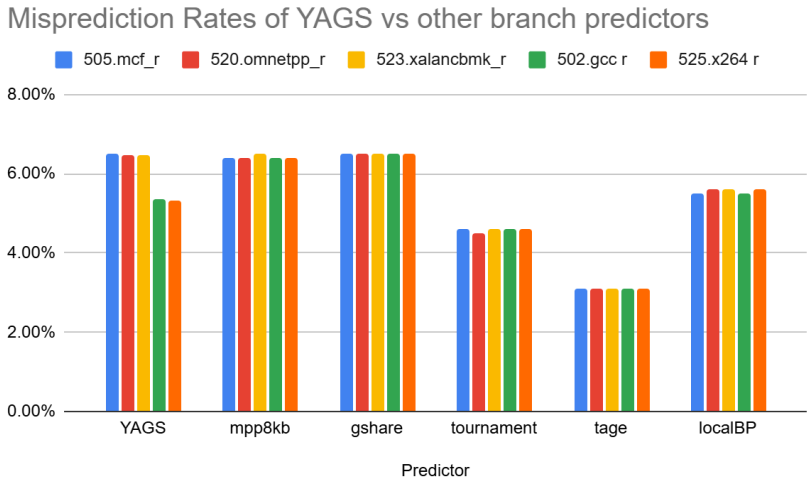


Figure 1. Misprediction rates over a set of benchmarks for our YAGS implementation as well as other branch predictors.

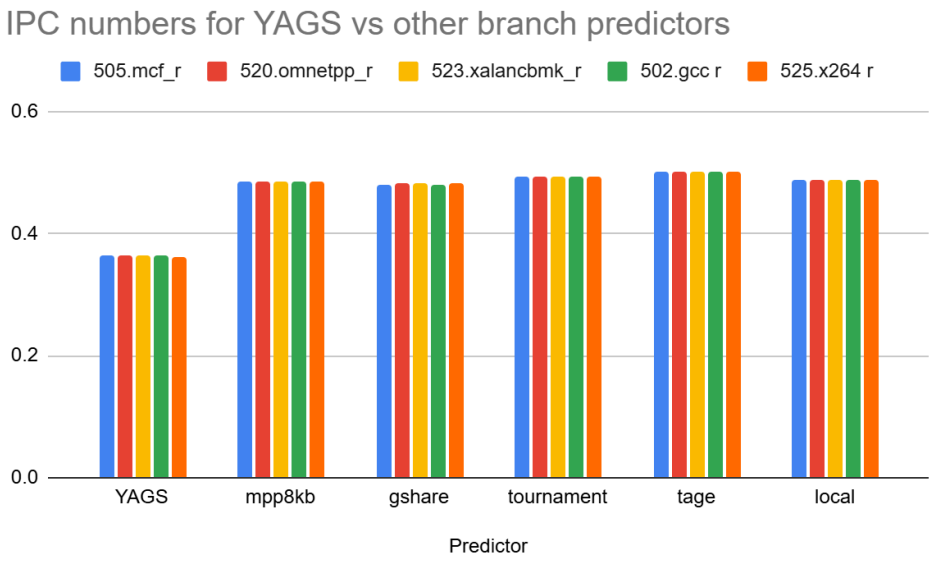


Figure 2. IPC over a set of benchmarks for our YAGS implementation, as well as other branch predictors.

Discussion & Conclusion

Throughout this work, we implemented the YAGS branch predictor in Gem5 and evaluated it over a set of benchmarks. Based on the results of our evaluations, it is clear that our evaluation setup has been implemented incorrectly. We initially believed it was due to running only 1 billion warm-up instructions, but that doesn't explain why the IPC doesn't change significantly across benchmarks for each predictor. One reason that may be the case is that the CPU core switch logic after warm-up is complete may not have been implemented correctly, possibly giving us warm-up results instead of measurement results. All things considered, it is evident that our simulations don't align with what we would qualitatively expect from the branch predictors we tested across the different benchmarks, as the benchmarks vary significantly in the types of workloads they subject the predictors to. Further investigation is needed to fully understand the exact source of this problem. While the misprediction rates of YAGS are in line with the paper's results, we expected to find more variance in the data, given the differences in the workload's characteristics. As for the IPC, we expected the data to be greater than 1, given that we tested on an Out-of-Order CPU, further signaling an error in the benchmark setup.

While our simulation results do not reflect expected behavior due to the setup issues described above, we can qualitatively reason about expected YAGS performance. Benchmarks with large branch signatures and complex control flow, such as 502.gcc_r and 520.omnetpp_r, should benefit most from YAGS's tagged direction caches, as aliasing is most destructive in these workloads. Memory-bound benchmarks like 505.mcf_r have sparser branching, meaning branch predictor choice has less overall impact on IPC. Across all benchmarks, we would expect YAGS to outperform Gshare while falling short of TAGE, as TAGE's multi-table tagged architecture represents a fundamentally more powerful approach regardless of workload characteristics.

References

[1] A. N. Eden and T. Mudge, “The YAGS branch prediction scheme,” in Proceedings. 31st Annual ACM/IEEE International Symposium on Microarchitecture, Dec. 1998, pp. 69–77. doi: 10.1109/MICRO.1998.742770.